

DESARROLLO DE SOFTWARE & DEVOPS

Gestión y Resolución de Bugs en GitHub

Flujos de trabajo optimizados, estrategias de remediación rápida (hotfix) y mantenimiento continuo de la calidad del código.

Git Workflow

GitHub Actions

Code Review

Diagnóstico y Clasificación de Errores (Triage)

Aislamiento y Reproducibilidad

Creación de un entorno controlado para replicar la falla. Se redacta un **test unitario fallido** antes de aplicar cualquier modificación (TDD).

Etiquetado Estandarizado

Uso estricto de etiquetas en GitHub Issues (**bug**, **severity:high**, **needs-triage**) para priorizar el impacto en producción.

Trazabilidad del Problema

Asignación directa de responsables y vinculación de reportes de telemetría o logs del sistema al Ticket correspondiente.



Ramificación y Corrección (Branching Strategy)

Convención de Nombres de Rama

Aislamiento en ramas específicas según la naturaleza de la solución:
`fix/issue-404-login` o `hotfix/auth-leak`.

Commits Atómicos y Semánticos

Mensajes claros bajo la especificación de Conventional Commits (`fix:`
`correct user payload parsing`) facilitando el historial.

Cierre Automático mediante PR

Vinculación explícita en la descripción del Pull Request usando palabras clave como `Fixes #123` para automatizar la resolución.



Automatización de Pruebas con GitHub Actions

Ejecución de Checks Pre-Merge

Workflows automáticos que ejecutan **linters**, análisis estático de código y suites de pruebas unitarias en cada push a la rama de fix.

Pruebas de Regresión Integradas

Garantizar que la solución introducida no afecte otras funcionalidades preexistentes dentro del repositorio principal.

Ambientes de Pre-despliegue (Preview)

Generación automática de entornos efímeros para la validación visual y funcional por parte del equipo de QA.



Revisión de Código y Estrategias de Merge

Peer Review Obligatorio

Requisito de al menos una aprobación (**CODEOWNERS**) para prevenir la introducción accidental de nuevos errores.

Análisis de Seguridad y Deuda Técnica

Revisión de vulnerabilidades con herramientas como CodeQL antes de autorizar la integración con la rama principal.

Squash and Merge

Condensación de múltiples commits intermedios en un único commit limpio al fusionar en la rama **main** o **master**.



Despliegue de Hotfixes y Telemetría

Estrategia de Despliegue Canario

Liberación gradual del parche a un porcentaje reducido de usuarios para evaluar la estabilidad del sistema.

Monitoreo Post-Despliegue

Seguimiento en tiempo real de métricas críticas (**error rates**, latencia) usando plataformas de APM.

Mecanismo de Rollback Rápido

Automatización del revert en Git en caso de comportamientos anómalos impredecibles en el entorno de producción.



Referencias Bibliográficas

Chacon, S., & Straub, B. (2014). [Pro Git](https://git-scm.com/book/en/v2) (2.^a ed.). Apress. <https://git-scm.com/book/en/v2>

Fowler, M. (2018). [Refactoring: Improving the design of existing code](#) (2.^a ed.). Addison-Wesley Professional.

GitHub Docs. (2023). [About continuous integration with GitHub Actions](https://docs.github.com/en/actions). GitHub Documentation. <https://docs.github.com/en/actions>

Humble, J., & Farley, D. (2010). [Continuous delivery: Reliable software releases through build, test, and deployment automation](#). Addison-Wesley Professional.

Kim, G., Humble, J., Debois, P., & Willis, J. (2016). [The DevOps handbook: How to create world-class agility, reliability, and security in technology organizations](#). IT Revolution Press.